

LiquidAI / LFM2.5-Encoder-350M-Diffusion

like 8

Follow Liquid AI 4.42k

Text Generation
Transformers
Safetensors
English
lfm2
fill-mask
liquid
lfm2.5

bidirectional
masked-lm
encoder
diffusion-language-model
masked-diffusion
mdlm

instruction-tuned
conversational
custom_code
License: lfm1.0



Deploy

→ Copy to bucket NEW

Use this model



Model card

Files

xet



Community 1


[Try LFM](#) • [Docs](#) • [LEAP](#) • [Discord](#)

LFM2.5-Encoder-350M-Diffusion

A full fine-tune of [LFM2.5-Encoder-350M](#) as a masked-diffusion instruction model that generates text by iteratively unmasking tokens instead of decoding left to right.

The model was SFT-trained on [mlabonne/open-perfectblend](#), a dataset of roughly 1.39M conversations, for 3 epochs.

Masked diffusion is a natural extension of masked-language modeling: the model starts from masked answer tokens, repeatedly predicts all masked positions, fills the most confident tokens, and continues until the answer is complete.

Downloads last month
85


Safetensors

Model size 0.4B params

Tensor type F32 Chat template

Files info

Inference Providers NEW

Text Generation

This model isn't deployed by any Inference Provider.



Ask for provider support

Model tree for LiquidAI/LFM2.5-E...

Base model LiquidAI/LFM2.5-350M-Ba...



Finetuned LiquidAI/LFM2.5-En...


Finetuned (17) [this model](#)

Quantizations



1 model

Find more details about our encoders in our [blog post](#).

 **Demos:** Try this fine-tuned model running in a CPU-only Hugging Face space: [Masked-diffusion text generation](#) — run the encoder as a chatbot that generates text by iteratively unmasking instead of left to right.

Usage

Install the required packages:

```
pip install torch transformers
```

Run masked-diffusion text generation:

```
import torch
from transformers import AutoModelForMaskedLM

model_id = "LiquidAI/LFM2.5-Encoder-350M-Diffusion"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForMaskedLM.from_pretrained(model_id)

messages = [{"role": "user", "content": "Give me a recipe for chocolate cake."}]
prompt = tokenizer.apply_chat_template(messages, tokenize=True, add_generation_prompt=True)
inputs = tokenizer(prompt, return_tensors="pt")

num_new_tokens = 12
mask_id = tokenizer.mask_token_id
input_ids = torch.cat(
    [inputs.input_ids, torch.full((1, num_new_tokens), mask_id, dtype=torch.long)],
    dim=-1,
)

attention_mask = torch.ones_like(input_ids)

with torch.no_grad():
```

```

for _ in range(num_new_tokens):
    mask_positions = (input_ids[0] == mask_token_id)
    if len(mask_positions) == 0:
        break

    logits = model(input_ids=input_ids, inputs_embeds=inputs_embeds)
    logits[:, len(tokenizer):] = -torch.finfo(logits.dtype).max

    for token_id in tokenizer.all_special_ids:
        if token_id != tokenizer.eos_token_id:
            logits[:, token_id] = -torch.finfo(logits.dtype).max

    probs = logits.softmax(dim=-1)
    confidence, token_ids = probs.max(dim=-1)
    best = confidence.argmax()
    input_ids[0, mask_positions[best]] = token_ids[best]

generated = input_ids[0, inputs.input_ids.shape[1]:]
text = tokenizer.decode(generated, skip_special_tokens=True)
print(text.strip())

```

Contact

- Got questions or want to connect? [Join our Discord community](#).
- If you are interested in custom solutions with edge deployment, please contact [our sales team](#).

Citation

```

@article{liquidAI2026Encoders,
  author = {Liquid AI},
  title = {LFM2.5-Encoders: Fast at Long Context Lengths},
  journal = {Liquid AI Blog},
  year = {2026},
  note = {www.liquid.ai/blog/lfm2-5-encoders}
}

```

 System theme

TOS

Privacy

About

Careers



Models

Datasets

Spaces

Pricing

Docs